EXP.NO:	9	MEMORY ALLOCATION: FIRST FIT AND BEST FIT
DATE:	/ / 2025	

AIM

To implement First Fit and Best Fit memory allocation strategies and observe how each strategy allocates memory blocks to processes with minimum internal fragmentation.

PROCEDURE

1. Input number of memory blocks and their sizes.
2. Input number of processes and their sizes.
3. First Fit: Allocate the first block that is large enough for the process.
4. Best Fit: Allocate the smallest block that is large enough (minimum fragmentation).
5. Display allocation details with fragment size for each process.

PROGRAM 1 – First Fit

```
#include <stdio.h>
int main() {
    int m, n, i, j;
    printf("Enter number of memory blocks:\n"); scanf("%d", &m);
    int block[m], original[m];
    printf("Enter size of each block:\n");
    for(i=0;i<m;i++) { scanf("%d",&block[i]); original[i]=block[i]; }
    printf("Enter number of processes:\n"); scanf("%d", &n);
    int process[n];
    printf("Enter size of each process:\n");
    for(i=0;i<n;i++) scanf("%d",&process[i]);
    for(i=0;i<n;i++) {
        int flag=0;
        for(j=0;j<m;j++) {
            if(block[j]>=process[i]) {
                int fragment=block[j]-process[i];
                printf("Process %d (size %d) -> Block %d (size %d), Fragment=%d\n",
                       i+1,process[i],j+1,original[j],fragment);
                block[j]-=process[i]; flag=1; break;
            }
        }
        if(!flag) printf("Process %d (size %d) -> NOT ALLOCATED\n",i+1,process[i]);
    }
    return 0;
}
```

PROGRAM 2 – Best Fit

```
#include <stdio.h>
int main() {
    int m, n, i, j;
    printf("Enter number of memory blocks:\n"); scanf("%d", &m);
    int block[m], original[m];
    printf("Enter size of each block:\n");
    for(i=0;i<m;i++) { scanf("%d",&block[i]); original[i]=block[i]; }
    printf("Enter number of processes:\n"); scanf("%d", &n);
    int process[n];
    printf("Enter size of each process:\n");
    for(i=0;i<n;i++) scanf("%d",&process[i]);
    for(i=0;i<n;i++) {
        int bestIndex=-1;
        for(j=0;j<m;j++) {
            if(block[j]>=process[i])
                if(bestIndex==-1||block[j]<block[bestIndex]) bestIndex=j;
        }
        if(bestIndex!=-1) {
            int fragment=block[bestIndex]-process[i];
            printf("Process %d (size %d) -> Block %d (size %d), Fragment=%d\n",
                   i+1,process[i],bestIndex+1,original[bestIndex],fragment);
            block[bestIndex]-=process[i];
        } else printf("Process %d (size %d) -> NOT ALLOCATED\n",i+1,process[i]);
    }
    return 0;}
```

OUTPUT

```
--- FIRST FIT ---
$ gcc firstfit.c && ./a.out
Enter number of memory blocks: 5
Enter size of each block: 100 500 200 300 600
Enter number of processes: 4
Enter size of each process: 212 417 112 426
Process 1 (size 212) -> Block 2 (size 500), Fragment=288
Process 2 (size 417) -> Block 5 (size 600), Fragment=183
Process 3 (size 112) -> Block 2 (size 288), Fragment=176
Process 4 (size 426) -> NOT ALLOCATED

--- BEST FIT ---
$ gcc bestfit.c && ./a.out
Enter number of memory blocks: 5
Enter size of each block: 100 500 200 300 600
Enter number of processes: 4
Enter size of each process: 212 417 112 426
Process 1 (size 212) -> Block 4 (size 300), Fragment=88
Process 2 (size 417) -> Block 2 (size 500), Fragment=83
Process 3 (size 112) -> Block 3 (size 200), Fragment=88
Process 4 (size 426) -> Block 5 (size 600), Fragment=174
```

RESULT

First Fit and Best Fit memory allocation algorithms were successfully implemented. Best Fit allocated all four processes with lower fragmentation compared to First Fit, demonstrating its advantage in reducing memory wastage.